Foglio di esercizi 7

Analisi Numerica, a.a. 2019-2020

Dipartimento di Scienze Matematiche Fisiche e Informatiche
Università degli Studi di Parma

Docente:
Prof.ssa Alessandra Aimi

Tutor:
Ariel S. Boiardi *

Esercizio 1: Consideriamo un generico problema di Cauchy

$$\begin{cases} y'(t) = f(t, y) & t \in (T_0, T_f) \\ y(T_0) = y_0 \end{cases} \tag{1}$$

per cui cerchiamo una soluzione approssimata sui nodi di una griglia uniforme di N sottointervalli su (T_0, T_f).

Essendo noti il valore iniziale della soluzione $y(T_0) = y_0$ e la sua pendenza in ogni punto del piano (t, y), data da $y'(t) = f(t, y)$, ad ogni passo di integrazione possiamo pensare di *far avanzare* la soluzione approssimandola, fra due nodi consecutivi, con un segmento allineato con le linee di flusso della ODE, come illustrato in fig. 1: questa è l'idea del metodo di Eulero:

$$y(t_{i+1}) \approx y(t_i) + hf(t_i, y(t_i)), \quad i = 0, \dots, N. \tag{2}$$

Per il metodo di Heun, ad ogni passo, usiamo la pendenza calcolata sulla attuale approssimazione mediata con la pendenza nell'approssimazione successiva, calcolata con la formula di Eulero:

$$y(t_{i+1}) \approx y(t_i) + \frac{h}{2}\left(f(t_i, y(t_i)) + f(t_{i+1}, \underbrace{y(t_i) + hf(t_i, y(t_i))}_{(2)\approx y(t_{i+1})}) \right), \quad i = 0, \dots, N. \tag{3}$$

Nel metodo di Eulero modificato invece la pendenza viene calcolata nel punto medio fra i due nodi successivi, usando l'approssimazione attuale mandata avanti di mezzo passo temporale con il metodo di Eulero:

$$y(t_{i+1}) \approx y(t_i) + hf\left(t_i + \frac{h}{2}, y(t_i) + \frac{h}{2} f(t_i, y(t_i))\right), \quad i = 0, \dots, N. \tag{4}$$

*Per dubbi e segnalazioni: `arielsurya.boiardi@studenti.unipr.it`

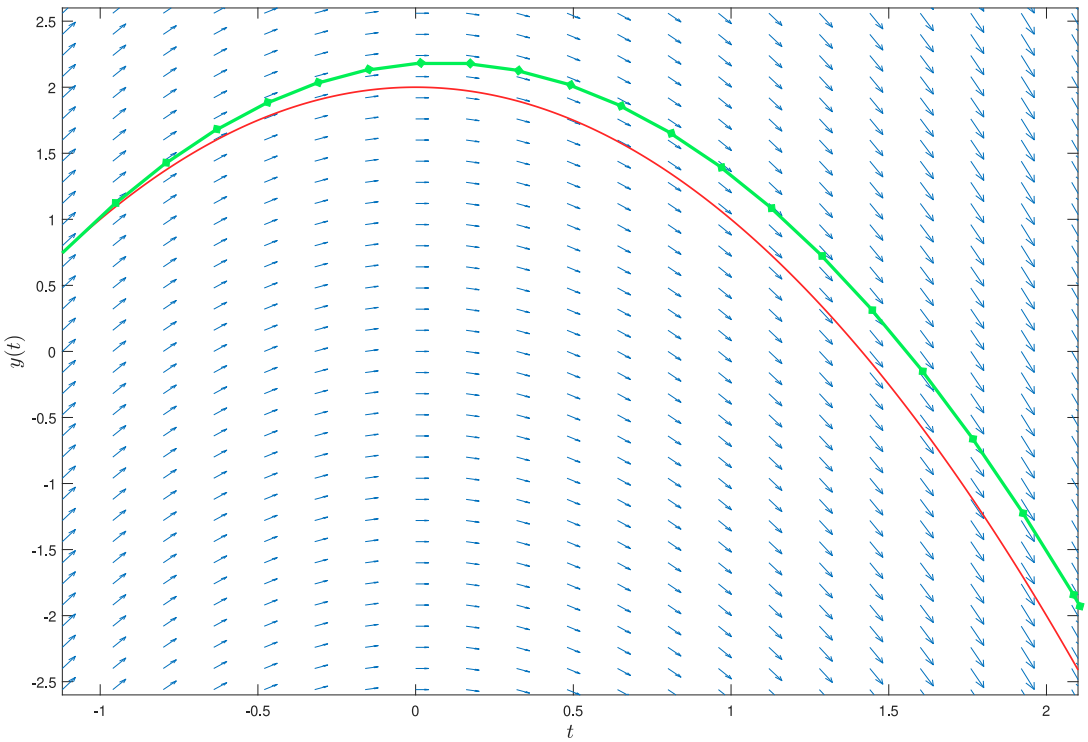


Figura 1: Le linee di flusso del campo $(1, -2t)$ rappresentano la ODE $y'(t) = -2t$, che in ogni punto del piano assegna il vettore tangente alla soluzione. Immersa nel campo è rappresentata (linea rossa) la soluzione della ODE con condizione iniziale[1] $y(-1.12) = 0.7456$, cioè $y(t) = 2 - t^2$. Abbiamo poi rappresentato in verde diversi passi del metodo di Eulero: a partire da un punto seguiamo le linee di flusso lungo una retta fino al nodo successivo, e ripetiamo la procedura. Evidentemente, dopo alcuni passi, la soluzione approssimata inizia a discostarsi sensibilmente dalla soluzione esatta.

Abbiamo predisposto 3 functions MATLAB che implementano rispettivamente un passo del metodo di Eulero (2), di Heun (3) e di Eulero modificato (4) nei listings da 1 a 3

Listing 1: Un passo della formula di Eulero

```
function [t1,y1]=EU(t0,y0,h,f)
% EU esegue un passo di integrazione del metodo di Eulero in avanti
%
% [t1,y1]=EU(t0,y0,h,f) restituisce ascissa e ordinata di un passo del
% metodo di Eulero in avanti a partire dal punto (t0,y0) con passo di
% discretizzazione h e forzante f.
%
% Input:
%   t0: instante iniziale
%   y0: valore iniziale
%   h: passo della discretizzazione
%   f: forzante della ODE (function handle di due variabili)
% Output:
%   t1: istante successivo
%   y1: soluzione approssimata in t1

t1=t0+h;
y1=y0+h*f(t0,y0);
end
```

Listing 2: Un passo della formula di Heun

```
function [t1,y1]=HEU(t0,y0,h,f)
% HEU esegue un passo di integrazione del metodo di Heun
%
% [t1,y1]=HEU(t0,y0,h,f) restituisce ascissa e ordinata di un passo del
% metodo di Heun a partire dal punto (t0,y0) con passo di discretizzazione
% h e forzante f.

t1=t0+h;
[~,y11]=EU(t0,y0,h,f);
y1=y0+h*0.5*(f(t0,y0) + f(t1,y11));
end
```

Listing 3: Un passo della formula di Eulero modificata

```
function [t1,y1]=EUM(t0,y0,h,f)
% EUM esegue un passo di integrazione del metodo di Eulero modificato
%
% [t1,y1]=EUM(t0,y0,h,f) restituisce ascissa e ordinata di un passo del
% metodo di Eulero modificato a partire dal punto (t0,y0) con passo di
% discretizzazione h e forzante f.

t1=t0+h;
```

[1] In realtà la curva è stata costruita con condizione $y(0) = 2$, nel grafico è stata scelta una costruzione diversa solo per rendere più interessante l'illustrazione.

```
[t11,y11]=EU(t0,y0,0.5*h,f);
y1=y0+h*f(t11,y11);
end
```

Esercizio 2: Vogliamo applicare il metodo di Eulero in avanti per approssimare la soluzione del problema di Cauchy

$$\begin{cases} y'(t) = \gamma y(t)(m - y(t)), & t \in (0, T] \\ y(0) = \alpha \end{cases} \tag{5}$$

per $m = 1000, \gamma = 0.001, \alpha = 1, T = 20$, di cui sappiamo la soluzione esatta

$$y(t) = \frac{m}{e^{-m\gamma t \frac{m-\alpha}{\alpha}} + 1}.$$

Come già anticipato nell'esercizio precedente in fig. 1, a partire dal valore iniziale assegnato, ogni passo del metodo di Eulero permette di far progredire la soluzione al nodo successivo, fino ad arrivare all'altro capo del dominio. Il metodo di Eulero per una griglia di passo costante arbitrario è implementato in listing 4 abbiamo scelto di implementare il metodo in modo da poter scegliere il numero di passi e l'ampiezza di ogni passo. Un'alternativa potrebbe essere quella di assegnare gli estremi del dominio e il numero di passi (pari al numero di sottointervalli) della discretizzazione.

Listing 4: Implementazione del metodo di Eulero in avanti

```
function [y,t]=eulero(t0,y0,f,h,N)
% EULERO metodo di Eulero per ODE
%
% [y,t]=EULERO(t0,y0,f,h,N) restituisce la soluzione approssimata y nei
% punti t, nodi di una griglia uniforme di N sottointervalli di ampiezza h.
%
% Input:
%   t0,y0: nodo e valore inizale
%   f: forzante della ODE (funzione handle f(t,y))
%   h: passo della discretizzazione
%   N: numero di intervalli della griglia
%
% Output:
%   y: valori della soluzione approssimata sui nodi
%   t: nodi della griglia
%
% Note:
%   - Abbiamo messo come secondo output la griglia perche' puo' tornare
%   utile per i grafici. La abbiamo messa al secondo putput perche' se non
%   serve non c'e' da ricordarsene, basta assegnare un solo output e il
%   secondo verra' trascurato.
t=zeros(1,N+1);
y=zeros(1,N+1);
for idx=0:N-1
    t(idx+1)=t0;
    y(idx+1)=y0;
```

```
    [t1,y1]=EU(t0,y0,h,f);
    t0=t1;
    y0=y1;
end
t(N+1)=t1;
y(N+1)=y1;
end
```

Il resto dell'esercizio si svolge interamente in MATLAB , una soluzione è illustrata nel listing 5.

Listing 5: Script per la risoluzione dell'esercizio 2

```
%% Soluzione esercizio 2 WS7
clear; close all; clc
%% Parametri e soluzione esatta
% Consideriamo il problema di Cauchy
%       y'= gamma*y*(m-y) per t=(0,T]
%       y(0)=alpha
% con i seguenti parametri:
m=1000;
gamma=0.001;
alpha=1;
T0=0;
T=20;

f=@(t,y) gamma*y.*(m-y);

yex=@(t) m./(exp(-m*gamma*t)*((m-alpha)/alpha)+1);

tt=0:0.01:T;
plot(tt,yex(tt))
hold on
xlabel("$t$",'interpreter','latex','FontSize',14)
ylabel("$y(t)$",'interpreter','latex','FontSize',14)
legend_content{1}="Soluzione esatta";

%% Approssimazione
% Iniziamo con una approssimazione grossolana:
h=1;
% dovremo fare un numero di passi che permetta di coprire tutto
% l'intervallo (0,T]:
N=T/h;
% Procediamo con il metodo di Eulero in avanti:
[y1,t1]=eulero(T0,alpha,f,h,N);
plot(t1,y1)
legend_content{2}=strcat("Eulero $h=",string(h),"$");

% Proviamo ora ad approssimare piu' finemente
h=0.5;
N=T/h;
[y1,t1]=eulero(T0,alpha,f,h,N);
plot(t1,y1)
legend_content{3}=strcat("Eulero $h=",string(h),"$");
```

```
% ... e ancora piu' finemente
% In questo caso attenzione a scegliere un h in modo che (T0-T)/h sia
% intero. Eventualmente si potrebbe prendere il risultato della divisione
% intera e incrementarlo di 1.
h=0.05;
N=T/h;
[y1,t1]=eulero(T0,alpha,f,h,N);
plot(t1,y1)
legend_content{4}=strcat("Eulero $h=",string(h),"$");

legend(legend_content, ...
    'interpreter','latex', 'FontSize',12, 'Location','best')

%% Massimo errore assoluto
% Vogliamo ora approssimare lo stesso problema a vari livelli di finezza e
% vedere l'andamento dell'errore al crescere dei nodi della griglia:
for idx=1:19
    N=2^idx*10;      % Numero di sottointervalli della griglia
    NN(idx)=N;
    H=2^(1-idx);     % Si potrebbe calcolare T/N, ma lo abbiamo esplicito
    HH(idx)=H;
    % Calcoliamo ora le approssimazioni sulla griglia con metodo di Eulero
    [yH,tH]=eulero(T0,alpha,f,H,N);
    % Confrontiamo ora l'approssimazione ottenuta con la soluzione esatta
    % sui nodi
    ErrPt=abs(yH-yex(tH));
    % Il massimo dell'errore e' semplicemente
    Err(idx)=max(ErrPt);
end
% In una nuova figura andiamo a fare vedere la convergenza. Per riuscire a
% vedere qualcosa plottiamo in scala logaritmica
figure
loglog(NN,Err,'.-','MarkerSize',10)
title("Grafico di convergenza metodo di Eulero",...
    'interpreter','latex','FontSize',14);
xlabel("Numero di sottointervali $N_i=2^i 10,\ i=1,\ldots,19$",...
    'interpreter','latex','FontSize',12)
ylabel("$e_i=max_{j=0,\ldots, N_i}|y_j-y(t_j)|$",...
    'interpreter','latex', 'FontSize',12)
```

Come possiamo vedere in fig. 2 le approssimazioni ottenute con il metodo di Eulero presentano un ritardo rispetto alla soluzioni esatta, che però viene sensibilmente diminuito raffinando la discretizzazione. Questo fenomeno è dato dal fatto che le soluzioni sono calcolate successivamente, e ad ogni passo l'errore tende ad accumularsi (vedere anche l'esempio in fig. 1). Il grafico di convergenza in fig. 3a conferma che l'errore massimo commesso (almeno sui nodi) diminuisce al crescere del numero di nodi e raggiunge il minimo sulla griglia più fine ($i = 19$).

Nella lezione *Approfondimenti* è stato mostrato per lo stesso esempio che il minimo il minimo errore si ottiene per $i = 11$, in quanto per griglie più fini l'errore di arrotondamento in aritmetica di macchina può diventare dominate sull'errore di tron-

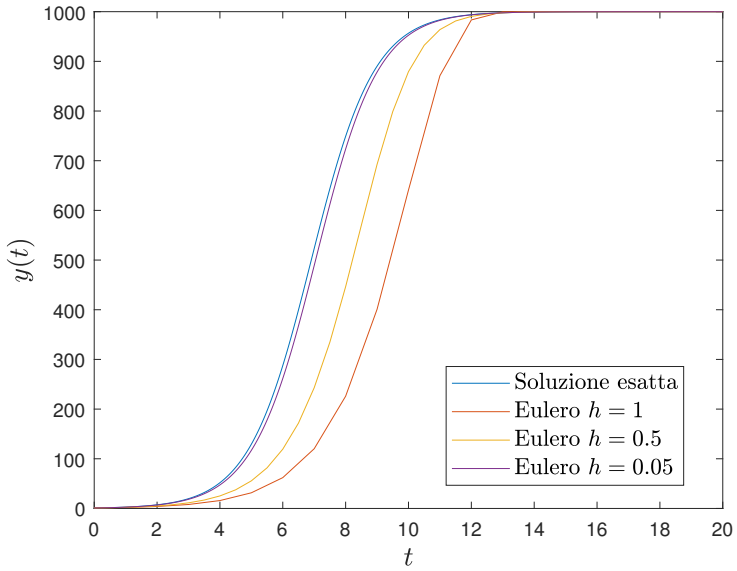


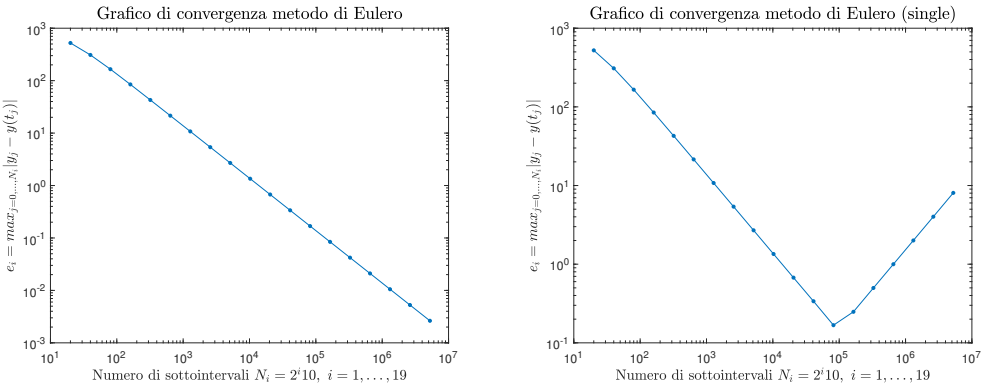
Figura 2: Soluzione esatta del problema di Cauchy (5) e sua approssimazioni ottenute con il metodo di Eulero in listing 4.

camento dovuto al metodo numerico e quindi far cessare la validità del teorema di convergenza. L'errore di arrotondamento in aritmetica di macchina dipende però da numerosi fattori fra cui (e non esaustivamente) la macchina stessa, il sistema di calcolo e l'implementazione del codice. Siccome MATLAB lavora automaticamente in doppia precisione, riusciamo a tenere controllati gli errori di arrotondamento in modo che non inficino il teorema di convergenza. Possiamo però riprodurre un comportamento simile a quello mostrato a lezione costringendo MATLAB a a lavorare in semplice precisione: basta dare uno degli input in formato `single`, a quel punto MATLAB passa automaticamente a fare tutti i calcoli in semplice precisione. Nel listing 5 sostituendo ad esempio `gamma=single(0.001)`, ogni volta che viene richiesta una valutazione di `f`, cioè in ogni passo del metodo di Eulero, il calcolo (e quindi il risultato) passa a singola precisione. Con questa rappresentazione numerica meno precisa otteniamo il grafico di convergenza in fig. 3b, nel quale possiamo osservare un aumento dell'errore dopo $i = 13$.

Esercizio 3: Iniziamo con l'implementare, analogamente a quanto fatto nell'esercizio 2, il metodo di Heun in listing 6 ed il metodo di Eulero modificato in listing 7. Come prima si tratta soltanto fare una successioni di passi di `HEU` o `EUM` rispettivamente, fino ad aver ricostruito la soluzione su tutti i punti della griglia.

Listing 6: Implementazione del metodo di Heun

```
function [y,t]=heun(t0,y0,f,h,N)
% HEUN metodo di Heun per ODE
```



(a) Convergenza in doppia precisione. (b) Convergenza in semplice precisione.

Figura 3: Grafici di convergenza delle soluzioni del problema (5) ottenute con il metodo di Eulero in avanti in semplice e doppia precisione.

```
%
% [y,t]=HEUN(t0,y0,f,h,N) restituisce la soluzione approssimata y nei
% punti t, nodi di una griglia uniforme di N sottointervalli di ampiezza h.
%
% Input:
%   t0,y0: nodo e valore inizale
%   f: forzante della ODE (funzione handle f(t,y))
%   h: passo della discretizzazione
%   N: numero di intervalli della griglia
%
% Output:
%   y: valori della soluzione approssimata sui nodi
%   t: nodi della griglia
t=zeros(1,N+1);
y=zeros(1,N+1);
for idx=0:N-1
    t(idx+1)=t0;
    y(idx+1)=y0;
    [t1,y1]=HEU(t0,y0,h,f);
    t0=t1;
    y0=y1;
end
t(N+1)=t1;
y(N+1)=y1;
end
```

Listing 7: Implementazione del metodo di Eulero modificato

```
function [y,t]=eulero_mod(t0,y0,f,h,N)
% EULERO_MOD metodo di Eulero modificato per ODE
%
% [y,t]=EULERO_MOD(t0,y0,f,h,N) restituisce la soluzione approssimata y nei
% punti t, nodi di una griglia uniforme di N sottointervalli di ampiezza h.
```

```
%
% Input:
%   t0,y0: nodo e valore inizale
%   f: forzante della ODE (funzione handle f(t,y))
%   h: passo della discretizzazione
%   N: numero di intervalli della griglia
%
% Output:
%   y: valori della soluzione approssimata sui nodi
%   t: nodi della griglia

t=zeros(1,N+1);
y=zeros(1,N+1);
for idx=0:N-1
    t(idx+1)=t0;
    y(idx+1)=y0;
    [t1,y1]=EUM(t0,y0,h,f);
    t0=t1;
    y0=y1;
end
t(N+1)=t1;
y(N+1)=y1;
end
```

Consideriamo ora il problema di Cauchy

$$\begin{cases} y'(t) = -y(t) + 1, & t \in [0,1] \\ y(0) = 0 \end{cases} \tag{6}$$

per cui possiamo ricavare analiticamente la soluzione

$$y(t) = -e^{-t} + 1, \quad t \in [0,1], \tag{7}$$

possiamo quindi usarlo come benchmark confrontando le soluzioni ottenute con i metodi appena implementati con la soluzione esatta.

Nello spirito di proporre un'esplorazione dei risultati, più che i risultati stessi, abbiamo predisposto uno script (listing 8) nel quale è possibile ottenere molte informazioni in modo leggibile direttamente in MATLAB e confrontare i risultati.

Listing 8: Script per la risoluzione dell'esercizio 3

```
%% Esercizio 3 WS7
% Questo script non e' da considerarsi pienamente automatico. Sicuramente
% ci sono decine di interferenze fra le diverse sezioni. Serve piu' che
% altro per dare un'idea su come svolgere i singoli aspetti dell'esercizio
% e come fare un'aplorazione dei risultati che sia anche utilizzabile.
% Cambiando qualche parametro e' possibile che escano degli errori...
clear; close all; clc;
%% Dati del problema
% Abbiamo il problema di Cauchy
%       y' = - y + 1 su [0,1]
%       y(0)=0
```

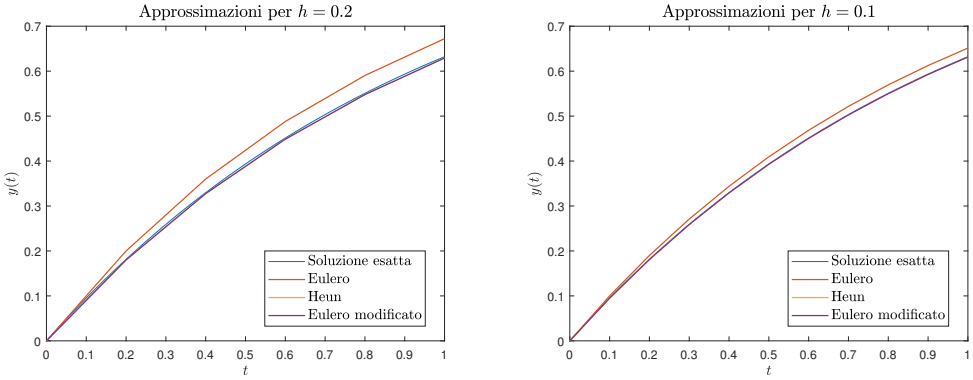
```
T0=0;
Tf=1;
f=@(t,y) -y + 1;
y0=0;

yex=@(t) - exp(-t) + 1;
t=T0:0.01:1;
plot(t,yex(t))
hold on

%% Confronto pari passo
% Fissato il passo di discretizzazione h, la seguente sezione confronta
% graficamente le soluzioni ottenute con i tre diversi metodi rispetto alla
% soluzione esatta. Produce poi una tabella per un confronto piu'
% quantitiativo in punti fissati della griglia, chiamati tQ.

h=0.1;
N=(Tf-T0)/h;

% Calcolo delle approssimazioni con lo stesso passo
[yEU,tEU]=eulero(T0,y0,f,h,N);
[yHEU]=heun(T0,y0,f,h,N);
[yEUM]=eulero_mod(T0,y0,f,h,N);

% Confrotiamo graficamente i risultati
plot(tEU,yEU,'LineWidth',1)
plot(tEU,yHEU,'LineWidth',1)
plot(tEU,yEUM,'LineWidth',1)

title(strcat("Approssimazioni per $h=",string(h),"$"),...
    'interpreter','latex','FontSize',14);
legend("Soluzione esatta","Eulero","Heun","Eulero modificato",...
    'interpreter','latex','FontSize',12,'Location','best');
xlabel("$t$",'interpreter','latex','FontSize',12)
ylabel("$y(t)$",'interpreter','latex','FontSize',12)
xlim([T0,Tf])

%% Tabella puntuale a pari passo
% Passiamo ora al confronto puntuale su alcuni punti, naturalmente abbiamo
% le approssimazioni solo sui punti della griglia, trascureremo quindi i
% punti in cui viene richiesta la soluzione che non sono presenti nella
% griglia. Iniziamo scegliendo i punti che ci interessano, in questo caso
tQ=0.1:0.1:1;

% Usando l'indicizzazione logica in MATLAB: usando ismembertol troviamo
% quali dei punti tQ sono nella griglia (come da documentazione) iQ sara'
% un vettore logico della dimensione di tEU con flase per tutti gli
% elementi di tEU che non compaiono in tQ, e true altrove. Osserviamo che
% questo approccio funziona bene nel caso (come in questo) non ci siano
% ripetizioni. Altra osservazione: usiamo ismembertol perche' abbiamo visto
% che semplicemente ismember ha problemi dati dall'aritmetica di macchina
% (vedere l'help per dettagli)
```

```
iQ=ismembertol(tEU,tQ);
% Trascuriamo, analogamente, i punti di tQ che non sono presenti nella
% griglia (eventualmente la tabella sara' vuota)
tQ=tQ(ismembertol(tQ,tEU));

% Ora possiamo costruire la tabella
Tpasso=table(tQ',yEU(iQ)',yHEU(iQ)',yEUM(iQ)',yex(tQ)');
Tpasso.Properties.VariableNames={'tQ','Eulero','Heun',...
    'Eulero_modificato', 'Soluzione_esatta'};

%% Confronto a pari costo
% Siccome il metodo eulero richiede una sola valutazione funzionale, mentre
% heun e eulero_mod ne richiedono 2 ad ogni iterazione, un modo per fare un
% confronto a pari costo e' semplicemente dimezzare il passo per eulero:
H=0.1;

hEU=H*0.5;
NEU=(Tf-T0)/hEU;
hHEU=H;
NHEU=(Tf-T0)/hHEU;
hEUM=H;
NEUM=(Tf-T0)/hEUM;

% Costruiamo ora le approssimazioni, dovremo salvare il vettore di nodi per
% tutti i metodi in modo da essere sicuri di usare quello giusto.
[yEU,tEU]=eulero(T0,y0,f,hEU,NEU);
[yHEU,tHEU]=heun(T0,y0,f,hHEU,NHEU);
[yEUM,tEUM]=eulero_mod(T0,y0,f,hEUM,NEUM);

figure
plot(t,yex(t));
hold on

% Confrotiamo graficamente i risultati
plot(tEU,yEU,'LineWidth',1)
plot(tHEU,yHEU,'LineWidth',1)
plot(tEUM,yEUM,'LineWidth',1)

title("Approssimazioni a pari costo",'interpreter','latex','FontSize',14);
legend("Soluzione esatta",...
    strcat("Eulero, $h=",string(hEU),"$"),...
    strcat("Heun, $h=",string(hHEU),"$"),...
    strcat("Eulero modificato, $h=",string(hEUM),"$"),...
    'interpreter','latex','FontSize',12,'Location','best');
xlabel("$t$",'interpreter','latex','FontSize',12)
ylabel("$y(t)$",'interpreter','latex','FontSize',12)
xlim([T0,Tf])

%% Tabella a pari costo
% Anche a pari costo possiamo costruire una tabella che confronti
% puntualmente le soluzioni sui pounti di tQ presenti in tutte le griglie.
% Siuccome la griglia per eulero e' diversa dalle altre, bisogna ricavare
% due set di indici...
```

```
iQEU=ismembertol(tEU,tQ);
iQHEU=ismembertol(tHEU,tQ);
% Siccome la griglia meno fine e' quella per heun, basta togliere da tQ i
% nodi della griglia che non appaio nella griglia per il metodo di Heun
tQ=tQ(ismembertol(tQ,tHEU));

Tcosto=table(tQ',yEU(iQEU)',yHEU(iQHEU)',yEUM(iQHEU)',yex(tQ)');
Tcosto.Properties.VariableNames={'tQ','Eulero','Heun',...
    'Eulero_modificato', 'Soluzione_esatta'};


%% Convergenza
% Tutti i confronti precedenti non tengono conto del dato piu' importante,
% cioe' la velocita' di abbattimento dell'errore, cerchiamo quindi di
% costruire un grafico di convergenza e dedurre l'ordine di convergenza
N=10;
for idx=0:19
    Ndx=N*2^idx;
    hdx=(Tf-T0)/Ndx;
    NN(idx+1)=Ndx;
    % Ora approssimiamo con i tre metodi e salviamo il massimo errore
    % commesso su tutto l'intervallo
    [yEU,tEU]=eulero(T0,y0,f,hdx,Ndx);
    ErrEU(idx+1)=max(abs(yEU-yex(tEU)));
    [yHEU,tHEU]=heun(T0,y0,f,hdx,Ndx);
    ErrHEU(idx+1)=max(abs(yHEU-yex(tHEU)));
    % Siccome abbiamo visto, e spiegato perche', HEU e EUM sono uguali,
    % risparmimo un po' di temp onelle iterazioni.
end

figure
loglog(NN,ErrEU,'.-','MarkerSize',10)
hold on
loglog(NN,ErrHEU,'.-','MarkerSize',10)

title("Grafico di convergenza",'interpreter','latex','FontSize',14)
xlabel("Numero di sottointervalli, $N_i=N2^i,\ i=0,\ldots,19$",...
    'interpreter','latex','FontSize',12)
ylabel("$e_i=max_{j=0,\ldots, N_i}|y_j-y(t_j)|$",...
    'interpreter','latex', 'FontSize',12)
legend("Eulero", "Heun",'interpreter','latex','FontSize',12,...
    'Location','best')
```

Nella prima parte confrontiamo i risultati ottenuti con i tre metodi con lo stesso passo di discretizzazione, troviamo i risultati confrontati sui nodi $t_i = 0.1i$, $i = 1, \ldots, 10$ nella tabella `Tpasso`:

```
>> Tpasso

        Tpasso =

        10x5 table
```



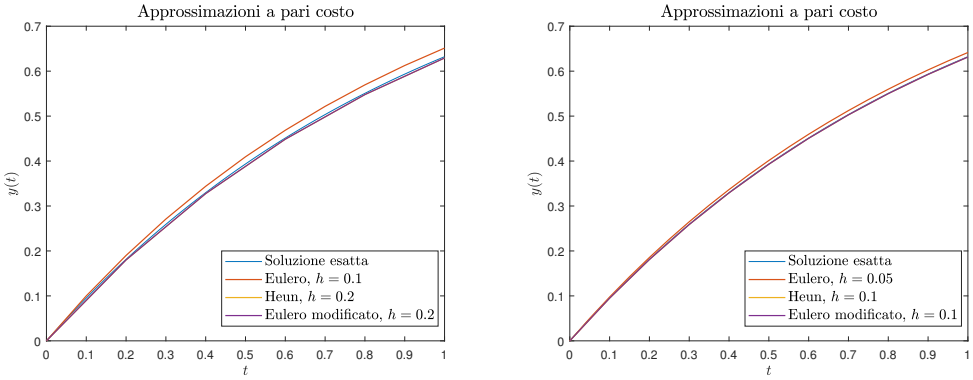
(a) (b)

Figura 4: Confronto dei tre metodi a pari passo. Osserviamo che il metodo di Heun e il metodo di Eulero modificato forniscono lo stesso risultato, come visto anche sui punti campionati. Ancora una volta, come prevedibile, il risultati peggiora allontanandosi dal punto iniziale.

tQ	Eulero	Heun	Eulero_modificato	Soluzione_esatta
0.1	0.1	0.095	0.095	0.09516
0.2	0.19	0.18097	0.18097	0.18127
0.3	0.271	0.25878	0.25878	0.25918
0.4	0.3439	0.3292	0.3292	0.32968
0.5	0.40951	0.39292	0.39292	0.39347
0.6	0.46856	0.4506	0.4506	0.45119
0.7	0.5217	0.50279	0.50279	0.50341
0.8	0.56953	0.55002	0.55002	0.55067
0.9	0.61258	0.59277	0.59277	0.59343
1	0.65132	0.63146	0.63146	0.63212

Possiamo facilmente osservare che il metodo di Heun e il metodo di Eulero modificato forniscono gli stessi risultati, almeno nei punti t_i considerati. In effetti i grafici in fig. 4 confermano lo stesso fatto per due diverse scelte del passo di discretizzazione. In effetti l'espressione del problema (6) chiarisce facilmente il motivo: abbiamo detto che la differenza fra il metodo di Heun e il metodo di Eulero modficato è che nel primo caso nella funzione di iterazione usiamo una media delle valutazioni di f fra due approssimazioni successive, mentre nel secondo valutiamo la f nel punto medio fra due approssimazioni successive. Siccome la f in (6) è lineare in y, le due operazioni descritte sono esattamente equivalenti.

Nella seconda parte vogliamo mettere in relazione l'errore con il costo dell'approssimazione in termini di valutazioni funzionali della f. Siccome i metodi di Heun e Eulero modificato richiedono il doppio delle valutazioni rispetto al metodo di Eulero semplice, possiamo permetterci di utilizzare quest'ultimo co un passo di discretizzazione dimezza, spendendo lo stesso numero di valutazioni di f. I grafici in fig. 5



(a) (b)

Figura 5: Confronto dei tre metodi a pari costo. In questo caso però in il metodo di Eulero risulta molto più competitivo.

mostrano che in questo caso il metodo di Eulero fornisce un'approssimazione molto più corretta, come confermato nei punti campionati nella tabella `Tcosto`:

```
>> Tcosto

        Tcosto =

        5x5 table

        tQ     Eulero      Heun      Eulero_modificato    Soluzione_esatta
        ___    ________    _______   ___________________   ________________

        0.1     0.0975      0.095          0.095              0.09516
        0.2    0.18549    0.18097        0.18097              0.18127
        0.3    0.26491    0.25878        0.25878              0.25918
        0.4    0.33658     0.3292         0.3292              0.32968
        0.5    0.40126    0.39292        0.39292              0.39347
        0.6    0.45964     0.4506         0.4506              0.45119
        0.7    0.51233    0.50279        0.50279              0.50341
        0.8    0.55987    0.55002        0.55002              0.55067
        0.9    0.60279    0.59277        0.59277              0.59343
          1    0.64151    0.63146        0.63146              0.63212
```

Naturalmente i metodi di Heun e Eulero modificato non sono stati introdotti per il piccolo miglioramento osservato nei confronti precedenti rispetto al metodo di Eulero, ma per aumentare l'ordine di consistenza e quindi, almeno potenzialmente, di convergenza. Vediamo quindi l'andamento dell'errore raddoppiando ripetutamente il numero di sottointervalli della griglia (cioè dimezzando il passo): fissato un numero iniziale N di sottointervalli, confrontiamo il massimo errore su tutti i nodi per i tre metodi[2], costruendo così il grafico di convergenza riportato in fig. 6. Si vede facilmente

[2]In realtà due, stante l'uguaglianza fra metodo di Heun e metodo di Eulero modificato.

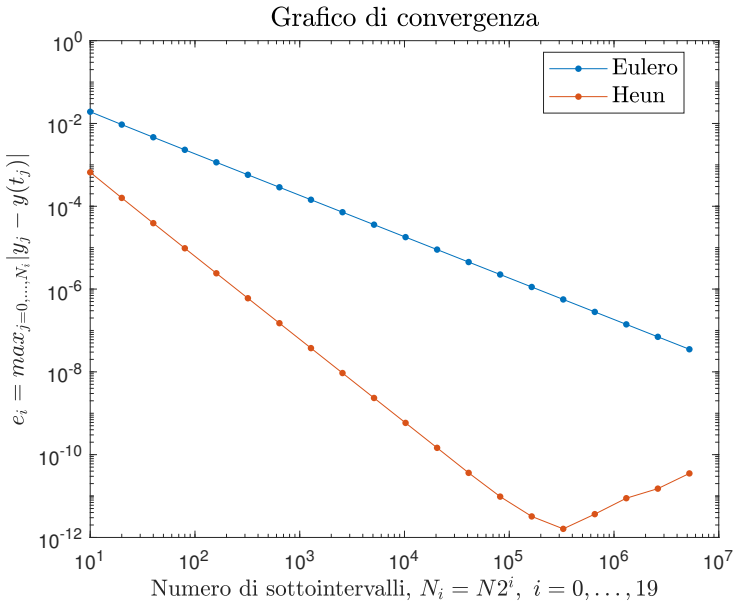


Figura 6: Grafico di convergenza dei metodi di Eulero e Heun.

che il metodo di Heun va a convergenza molto più rapidamente rispetto al metodo di Eulero.

Confrontando gli errori commessi nell'approssimazione sulle griglie via via dimezzate dell'esempio precedente possiamo anche stimare numericamente l'ordine del metodo. Chiamiamo E_h il massimo errore commesso sulla griglia con passo h e supponiamo che il metodo abbia ordine di convergenza p, ovvero

$$E_h \sim h^p.$$

Allora se dimezziamo il passo di discretizzazione avremo che

$$\frac{E_h}{E_{h/2}} = \frac{h^p}{(h/2)^p} = 2^p$$

passando agli esponenziali

$$e^{\log(E_h)-\log(E_{h/2})} = e^{p\log(2)}$$

e quindi

$$p = \frac{\log(E_h) - \log(E_{h/2})}{\log(2)}.$$

Possiamo quindi confermare che il metodo di Eulero è di ordine 1: siccome nel vettore `ErrEU` sono contenuti gli errori su griglie via via dimezzate, ogni elemento è l'errore a griglia dimezzata del precedente; per farla breve, gli ordini di convergenza sulle varie iterazioni saranno

```
>> (log(ErrEU(1:end-1))-log(ErrEU(2:end)))/log(2)

        ans =

        Columns 1 through 6

        1.0314    1.0154    1.0076    1.0038    1.0019    1.0009

        Columns 7 through 12

        1.0005    1.0002    1.0001    1.0001    1.0000    1.0000

        Columns 13 through 18

        1.0000    1.0000    1.0000    1.0000    1.0001    1.0002

        Column 19

        0.9982
```

Quindi confermiamo che il metodo di Eulero ha ordine 1. Ripetendo il ragionamento per il metodo di Heun

```
>> (log(ErrHEU(1:end-1))-log(ErrHEU(2:end)))/log(2)

        ans =

        Columns 1 through 6

        2.0552    2.0273    2.0136    2.0068    2.0034    2.0017

        Columns 7 through 12

        2.0008    2.0004    2.0002    2.0000    2.0015    2.0080

        Columns 13 through 18

        1.9055    1.5892    0.9978   -1.1801   -1.2796   -0.7674

        Column 19

        -1.2215
```

osserviamo che l'ordine di convergenza inizia a scostarsi da 2 in corrispondenza delle iterazioni in cui anche la pendenza del grafico in fig. 6 inizia a cambiare, e diventa negativo dove l'errore aumenta. Osserviamo che in effetti l'ordine di convergenza nelle varie iterazioni rappresenta la pendenza dei segmenti in fig. 6.

Esercizio 4: Per applicare il metodo di Eulero (e gli altri metodi implementati in questo foglio di esercizi) a sistemi di ODE basta costruire opportunamente la funzione f che descrive il sistema. Grazie alla flessibilità di MATLAB nella gestione di variabili

vettoriali, l'implementazione precedente si estende con correzioni minime a sistemi di ODE, come mostrato nel listing 9.

Listing 9: Metodo di Eulero adattato per sistemi di ODE

```
function [y,t]=eulero_vect(t0,y0,f,h,N)
% EULERO_VECT metodo di Eulero per sistemi di ODE
%
% [y,t]=EULERO_VECT(t0,y0,f,h,N) restituisce la soluzione approssimata Y
%  nei punti t, griglia uniforme di N sottointervalli di ampiezza h.
%
% Input:
%   t0,y0: nodo e valore inizale
%   f: struttura del sistema di ODE (come nelle routines ode di MATLAB)
%   h: passo della discretizzazione
%   N: numero di intervalli della griglia
%
% Output:
%   y: soluzione aprossimata sui nodi
%   t: nodi della griglia

t=zeros(1,N+1);
y=zeros(2,N+1);
for idx=0:N-1
    t(idx+1)=t0;
    y(:,idx+1)=y0;
    [t1,y1]=EU(t0,y0,h,f);
    t0=t1;
    y0=y1;
end
t(N+1)=t1;
y(:,N+1)=y1;
end
```

Consideriamo ora le equazioni di Lotka-Volterra

$$\begin{cases} \dot{x} = \alpha x + \beta xy, \quad \alpha > 0,\ \beta < 0 \\ \dot{y} = \gamma y + \delta xy \quad \gamma < 0,\ \delta > 0 \\ x(t_0) = x_0,\ y(t_0) = y_0. \end{cases} \tag{8}$$

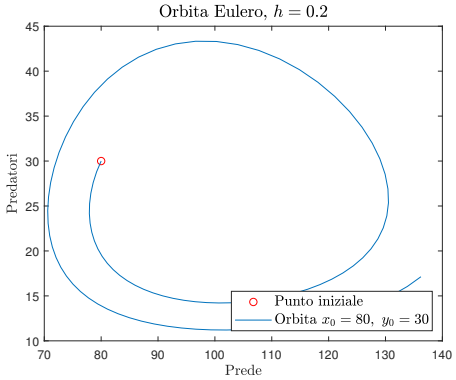
Possiamo riscrivere il sistema in forma vettoriale come

$$\begin{cases} \dot{Y}(t) = F(Y(t)), \quad t \in [t_0, t_f] \\ Y(t_0) = Y_0 \end{cases}$$

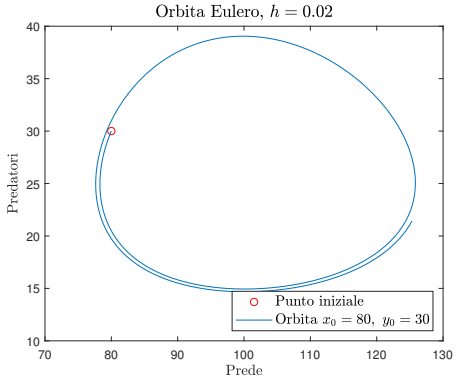
dove

$$Y(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}, \quad F\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} \alpha x + \beta xy \\ \gamma y + \delta xy \end{bmatrix}, \quad Y_0 = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}.$$

Allora la funzione F che descrive il sistema di Lotka-Volterra si implementa in MATLAB come



(a) L'orbita calcolata con $h = 0.2$ non si chiude.



(b) L'orbita calcolata con $h = 0.02$ tende a chiudersi.

```
function F=LV_fun(t,Y,alpha,beta,gamma,delta)
% Dinamica del sistema di Lotka-Volterra
F(1)=alpha*Y(1) + beta*Y(1)*Y(2);
F(2)=gamma*Y(2) + delta*Y(1)*Y(2);
end
```

Nel listing 10 abbiamo predisposto uno script MATLAB per fare uno po' di esperimenti sul sistema descritto dalle equazioni di Lotka-Volterra risolto con metodo di Eulero. Sappiamo che le equazioni di Lotka-Volterra hanno soluzioni periodiche, cioè danno luogo ad orbite chiuse nello spazio delle fasi, a patto di scegliere un intervallo temporale sufficientemente lungo. Vediamo però che una prima approssimazione ottenuta col metodo di Eulero (fig. 7a) non sembra volersi chiudere, anche se il tempo è decisamente sufficiente a farla chiudere, anzi, l'andamento spiraleggiante della traiettoria potrebbe anche farci erroneamente ipotizzare la presenza di un campo repulsivo all'interno della regione circondata dalla traiettoria. L'andamento divergente è però da imputare non alla dinamica del sistema, ma al metodo numerico utilizzato, infatti restringendo il passo di integrazione vediamo che il fenomeno tende a sparire(fig. 7b). Queste osservazioni mostrano quanto sia importante, nello studio *sperimentale* dei sistemi dinamici, avere ben presenti i limiti e le peculiarità dei metodi numerici utilizzati.

Listing 10: Esperimenti sul sistema Lotka-Volterra

```
%% Esercizio 4 WS7
clear; close all; clc
%% Dati del problema
% Consideriamo le euqazioni di Lotka-Volterra per la dinamica di due
% popolazioni (o specie chimiche, o altro) in interazione preda-predatore:
%   x'=alpha*x+beta*x*y
%   y'=gamma*y+delta*x*y
%   x(t0)=x0
%   y(t0)=y0
% con i parametri fissati
```

```
alpha=0.25;
beta=-0.01;
gamma=-1;
delta=0.01;

x0=80;
y0=30;
% Rappresentiamo il punto iniziale in modo da vedere da dove esce l'orbita
% che andremo a tracciare.
plot(x0,y0,'ro')
hold on
legend_info{1}="Punto iniziale";

% Siccome sappiamo che il diagramma di fase di questo sistema dinamico
% presenta degli andamenti ciclici aggiusteremo il tempo finale tf in modo
% da riuscre a compiere delle orbite complete.
t0=0;
tf=20;

% La funzione F che descrive la dinamica del sistema e' descritta nella
% subfunction LV_fun in fondo allo script (le subfucntions devono essere
% in fondo).

%% Discretizzazione
N=1000;
h=(tf-t0)/N;
[Y,t]=eulero_vect(t0,[x0,y0],@(t,Y) LV_fun(t,Y,alpha,beta,gamma,delta)...
    ,h,N);

% Possiamo confronta con ode45
% options=odeset('reltol',1e-8,'abstol',[1e-8, 1e-8]);
% Y=ode45(@(t,Y) LV_fun(t,Y,alpha,beta,gamma,delta)',t0:0.0001:tf,[x0,y0],
    options);
% Y=Y.y;

%% Rappresentazione
plot(Y(1,:),Y(2,:))
legend_info{2}=strcat("Orbita $x_0=",string(x0),",\ y_0=",string(y0),"$");
legend(legend_info,'interpreter','latex','Location','se','fontSize',12)
title(strcat("Orbita Eulero, $h=",string(h),"$"),'interpreter','latex',...
    'FontSize',14)
xlabel("Prede",'interpreter','latex','FontSize',12)
ylabel("Predatori",'interpreter','latex','FontSize',12)
%% Struttura del sistema dinamico
function F=LV_fun(t,Y,alpha,beta,gamma,delta)
% Dinamica del sistema di Lotka-Volterra
F(1)=alpha*Y(1) + beta*Y(1)*Y(2);
F(2)=gamma*Y(2) + delta*Y(1)*Y(2);
end
```